On-Device Intelligence for 5G RAN: Knowledge Transfer and Federated Learning Enabled UE-Centric Traffic Steering

Han Zhang, Hao Zhou, *Member, IEEE*, Medhat Elsayed, Majid Bavand, Raimundas Gaigalas, Yigit Ozcan, and Melike Erol-Kantarci, *Senior Member, IEEE*

***Abstract*—Traffic steering (TS) is a promising approach to support various service requirements and enhance transmission reliability by distributing network traffic loads to appropriate base stations (BSs). In conventional cell-centric TS strategies, BSs make TS decisions for all user equipment (UEs) in a centralized manner, which focuses more on the overall performance of the whole cell, disregarding specific requirements of individual UE. The flourishing machine learning technologies and evolving UE-centric 5G network architecture have prompted the emergence of new TS technologies. In this paper, we propose a knowledge transfer and federated learning-enabled UE-centric (KT-FLUC) TS framework for highly dynamic 5G radio access networks (RAN). Specifically, first, we propose an attention-weighted group federated learning scheme. It enables intelligent UEs to make TS decisions autonomously using local models and observations, and a global model is defined to coordinate local TS decisions and share experiences among UEs. Secondly, considering the individual UE's limited computation and energy resources, a growing and pruning-based model compression method is introduced, mitigating the computation burden of UEs and reducing the communication overhead of federated learning. In addition, we propose a Q-value-based knowledge transfer method to initialize newcomer UEs, achieving a jump start for their training efficiency. Finally, the simulations show that our proposed KT-FLUC algorithm can effectively improve the service quality, achieving 65% and 38% lower delay and 52% and 57% higher throughput compared with cell-based TS and other UE-centric TS strategies, respectively.**

***Index Terms*—Traffic steering, federated learning, on-device intelligence, transfer learning, model compression.**

I. Introduction

IN RECENT years, the explosive growth of traffic demand has pushed network operators to continuously improve their management strategies to meet diverse service requirements [1]. Traffic steering (TS) is an important network function that distributes network traffic load across multiple transmission paths. It makes user equipment (UE) connected to the best serving base station (BS) and choose best bands when multiple BSs provide coverage in the same geographical area [2]. Meanwhile, multiple radio access technologies (multi-RATs), dual connectivity (DC) and mobility also increase the complexity of TS management [3]. Therefore, how to manage network traffic efficiently has become a very challenging task for 5G beyond and 6G networks [4]. Conventional TS strategies are usually cell-centric, in which BSs make TS decisions for all the connected UEs. For example, [5] proposed a centralized load-balancing algorithm to decide handovers by calculating an offset value for users in the same cluster. However, such strategies fail to consider the unique demand of heterogeneous UEs with different traffic types and quality of service (QoS) requirements. Additionally, the conventional centralized method may increase the computation burden on the central controller, and lead to privacy and security concerns.

As the traffic types diversify, UE-centric TS strategies have appeared and attracted more attention. In a UE-centric TS, each UE selects the appropriate RAT, cell and frequency band that best serves their need. Compared with conventional cell-centric methods, the UE-centric scheme indicates that each UE can make its decision using local observations and experience, and then submit it to the BS for coordination. Hence, UE-centric networks with on-device intelligence have become a promising technique for TS [7], [8], [9], [11]. The benefits of designing a UE-centric network include:

(1) The UE-centric framework can mitigate the computation burden on the central controller because some computational tasks are transferred to the UEs.

(2) The UE-centric framework enables better QoS for diverse UEs because each UE can customize its TS demand and submit local decisions to BSs for coordination.

(3) Compared with conventional cell-centric frameworks, UE-centric schemes can better preserve user privacy because some local observations of UE can be locally kept and will not be shared with the centralized controller.

In addition, by leveraging on-device intelligence and offloading part of the processing computation to the UE side, the UE-centric TS framework also has better scalability

Manuscript received 21 April 2023; revised 21 September 2023 and 18 November 2023; accepted 27 November 2023. Date of publication 6 December 2023; date of current version 9 April 2024. This work has been supported by MITACS and Ericsson Canada, and NSERC Canada Research Chair programs. The associate editor coordinating the review of this article and approving it for publication was Y. Sagduyu. *(Corresponding author: Melike Erol-Kantarci.)*

Han Zhang, Hao Zhou, and Melike Erol-Kantarci are with the School of Electrical Engineering and Computer Science, University of Ottawa, Ottawa, ON K1N 6N5, Canada (e-mail: hzhan363@uottawa.ca; hzhou098@uottawa.ca; melike.erolkantarci@uottawa.ca).

Medhat Elsayed, Majid Bavand, Raimundas Gaigalas, and Yigit Ozcan are with the Networks R&D Department, Ericsson, Ottawa, ON K2K 2V6, Canada (e-mail: medhat.elsayed@ericsson.com; majid.bavand@ericsson.com; raimundas.gaigalas@ericsson.com; yigit.ozcan@ericsson.com).

Digital Object Identifier 10.1109/TCCN.2023.3340033

2332-7731 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on October 05,2026 at 18:03:27 UTC from IEEE Xplore. Restrictions apply.

compared to cell-centric TS. When the number of UE increases, the sum of on-device intelligence also increases. As a result, the framework is able to accommodate a very large number of UEs. Despite the potential benefits, UE-centric TS requires multiple distributed models rather than a single centralized model. The limited local observations of individual UE and coordination between multiple heterogeneous UEs lead to great complexity for the design and implementation. In addition, a single UE may have limited computational and energy resources for decision-making, increasing the difficulty of network management. In recent years, machine learning (ML) technologies have been widely applied in wireless communications to provide automated solutions for high-complexity tasks [12]. Especially, deep reinforcement learning (DRL)-based algorithms are proposed to deal with the increasing complexity of network environment. For instance, DRL has been used in [13] to allocate physical resource blocks and control the power level of UEs, and [14] applies DRL for joint user association and CU–DU placement under the O-RAN architecture. In [15] and [16], DRL is applied to dynamic resource allocations for 5G networks.

However, there are challenges in implementing UE-centric ML algorithms. Since each UE is considered an independent agent without direct information exchange with other UEs, it can only obtain limited information from local observations. Moreover, decentralized models may take long iterations to adapt to environmental changes, leading to a relatively long exploration time and slow convergence. In addition, every time a new UE is admitted to the network, it must start learning from the beginning, and this will degrade the overall network performance. To this end, we adopt an attention-weighted federated learning technique and use a global server to coordinate local models. In particular, the decentralized UEs upload the parameters of their local models to the global server. According to the performance of local models, an attention weight is calculated for each UE using a sigmoid function. The global server aggregates the local models into a global model by calculating the attention weights of the local models. Then the global parameters will be returned to the local model as feedback for model updates. Compared with conventional federated learning, the attention mechanism can improve model aggregation and obtain a better global model. Considering the advantages of accelerated convergence, federated learning can be widely used in distributed learning-based network control applications such as distributed resource allocation and edge caching.

Meanwhile, in a dynamic network environment, existing users may leave the network, and new users will join the network without any intelligent experience. Then these inexperienced new users may degrade the system performance, since they have no knowledge of TS strategies. To this end, a knowledge transfer approach is incorporated into the proposed framework. Specifically, knowledge acquired by existing users will be transferred to new users and give newcomers a good initialization for a quick start [18]. It will reduce the training efforts of new users, and mitigate performance degradation caused by inexperienced new UEs.

In addition, considering the large size and complex structure of ML models in most cases, training ML algorithms are supposed to be computation-intensive [19]. A UE may have limited computational resources for algorithm training and neural network updating, especially when running multiple applications simultaneously. Moreover, these computation-intensive tasks incur extra power consumption for the UE [31]. As a result, the deployed ML models are expected to be computationally resource-efficient. To address this issue, we proposed a growing and pruning-based model compression technique. By gradually splitting the most competitive neurons and eliminating the weakest neurons, we can estimate the smallest model size without degrading the TS performance.

Finally, inspired by these benefits and challenges, we propose a knowledge transfer and federated learning-enabled UE-centric (KT-FLUC) TS framework in conjunction with the concept of on-device intelligence. Different from the conventional TS schemes, each UE can make individual TS strategies by training ML models on the UE side. Through this structural innovation, our proposed framework has advantages in scalability, performance, and privacy protection. The main contributions are:

(1) We use several innovative techniques to solve the challenges of implementing UE-centric TS. We propose a unique transfer learning approach of providing expert q-values by building local expert models, which have been barely used in existing studies. We also combine grouping techniques and attention mechanisms with the conventional FL techniques to make it more suitable for our proposed scenario. We considered a time- variant network in which new UEs may arrive and leave the network dynamically [20].

(2) We design a KT-FLUC framework for UE-centric TS. Different from existing frameworks that simply bring different techniques together, in the proposed framework, these innovative techniques are inter-playing and mutually enhancing each other as a whole. In this work, the transfer learning technique makes the level of training of participants in federated learning relatively consistent. This process facilitates the cooperation between participants and thus contributes to FL. On the other hand, federated learning provides a suitable expert model for transfer learning through global aggregation and opens up potential applications for transfer learning. Working in conjunction with each other, each of these technologies can be made more useful in the designated scenario.

(3) We also propose a novel growing and pruning method to compress the DRL model and determine the optimal neural network (NN) size for TS. In this method, the neuron number of hidden layers of each UE is firstly increased and then decreased. The network performance under different model sizes is recorded, and the optimal number of hidden neurons can be established according to the performance decay rate with varying degrees of model compression. This method can effectively reduce the computation burden and power consumption on devices. To perform model compression more efficiently, we also propose a novel neuron splitting method, and this is the first time we apply model compression to a wireless communication application.

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on October 05,2026 at 18:03:27 UTC from IEEE Xplore. Restrictions apply.

The rest of this work is organized as follows. Section II introduces the related work. Section III shows the system model and problem formulation. Section IV explains the KT-FLUC framework and the model compression. The simulations are shown in Section V, and Section VI concludes this work.

II. Related Work

TS has been widely investigated under cell-centric frameworks. In [5], a centralized DRL framework is designed to decide the individual cell offset to control the handover of UEs. Reference [21] studied how to offload a mobile UE from macro cells to small cells and derived optimal offloading strategies with resource partitioning. In [23], a centralized radio access network (RAN) management module is designed to make TS decisions according to the traffic load of different RAT. In [24], a deep Q-network (DQN) is used to make traffic steering decisions with multiple RATs in a dynamic environment. In [25], a load-aware traffic steering algorithm is performed based on centralized hierarchical reinforcement learning.

These studies demonstrate that cell-centric TS can improve network efficiency and average network performance. However, in most existing works, there will be considerable overhead between the centralized controller and distributed UE. The centralized controller must process a large amount of information which will lead to a heavy computation burden. On the other hand, in a centralized architecture, the information submitted by the UEs may contain private information, such as traffic profiles that may infer UE locations and so on. The data exchange between the UEs and the controller makes the system more vulnerable to eavesdropping or attacks. As a result, privacy and security concerns are involved. Moreover, most of these frameworks fail to provide individual TS strategies for each UE, which may not satisfy the varying requirements of different UEs.

Meanwhile, some existing works investigate UE-centric frameworks with ML techniques. Reference [7] studies how to design a UE-centric network selection framework in ultra-dense heterogeneous networks by choosing the optimal RAT according to the link state prediction. Reference [26] provides a fully distributed TS approach running on distributed devices and optimizes each UE's received signal strength and energy consumption. In [27], distributed UEs choose their cooperative BS according to the received signal interference noise ratio (SINR) and predicted load of BS. Reference [28] designed a UE-based distributed TS mechanism between Wi-Fi networks and LTE with Q-learning. Although the above works adopt UE-centric frameworks and some use ML techniques such as federated learning, our work differs from existing works in three aspects. Firstly, most existing works simply deploy models on the UE side and fail to consider the limited resource of UEs. As a result, they may impose extra computation burdens on devices. Secondly, the collaboration between decentralized UEs is rarely considered in previous works, leading to low efficiency when applying the algorithm in practice. In addition, these works consider a static network environment with a fixed number of users, which is unrealistic, but our work includes a dynamic environment where UEs may join and leave the network. This work combines the concepts of reinforcement learning, federated learning, knowledge transfer, and model compression for the first time, merging the advantages of different techniques to collaboratively set up a UE-centric framework for TS.

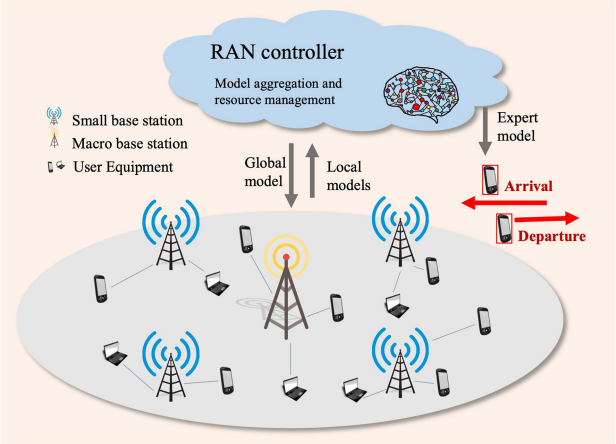


Fig. 1. Dynamic wireless networks with one MBS and several SBSs.

III. System Model and Problem Formulation

A. Architecture

The proposed TS system model of this work is shown in Fig. 1. We consider a network with two coexisting RATs, 5G and long-term evolution (LTE), with DC for downlink transmission. We assume the system has M UEs and N BSs. One BS is a macro BS (MBS) equipped with the LTE technology, and the remaining $(N-1)$ are small BSs (SBSs) equipped with the 5G technology. Each UE can choose a single BS at one time for better service. BSs generate interference with each other due to frequency reuse. Meanwhile, dynamic UE arrival and departure are involved, and we assume UEs arrive at the cell with a fixed file length to be transmitted [29].

We assume UEs have two types of services, guaranteed bit rate (GBR) service and non-GBR service, which have different quality of service (QoS) requirements [30]. Each UE either runs GBR traffic or non-GBR traffic. Most GBR services are sensitive to transmission delay, while non-GBR services require higher throughput [32]. So, the optimization goal of this TS framework is to minimize the packet delay of GBR services and maximize the throughput of non-GBR services.

B. Communication Model

The communication delay consists of two components, the queuing delay and the transmission delay:

$$d_m = d_m^{que} + d_m^{tx}, \tag{1}$$

where d_m^{que} is the queuing delay of the last packet transmitted to the user m. It refers to the delay incurred while the packet waits in the BS before being transmitted. d_m^{tx} is the transmission delay of the packet. It refers to the time pushing

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on October 05,2026 at 18:03:27 UTC from IEEE Xplore. Restrictions apply.

the bits onto the link, which is given as:

$$d_m^{tx} = \frac{L_m}{C_{n,m}}, \tag{2}$$

where L_m denotes the packet length and $C_{n,m}$ denotes the link capacity between the nth BS and the mth UE. According to Shannon's theorem, the link capacity is influenced by the signal interference noise ratio (SINR) between the BS and the UE, which can be formulated as:

$$C_{n,m} = \sum_{r \in R_n} B_r log_2\big(1 + \eta_{n,r,m}\big), \tag{3}$$

where R_n denotes the set of all the available resource blocks of the nth BS and B_r denotes the available bandwidth of the rth resource block. $\eta_{n,r,m}$ denotes the SINR between the transmission link between the nth BS and the mth UE with the rth resource block. It can be given as:

$$\eta_{n,r,m} = \frac{\alpha_{n,r,m} g_{n,m} P_{n,r}}{\sum_{n' \in N, n' \neq n} \sum_{m' \in M_{n'}} \alpha_{n',r,m'} g_{n',m} P_{n',r} + B_r N_0}, \tag{4}$$

where $\alpha_{n,r,m}$ is a binary indicator denoting whether the rth physical resource block of the nth BS is allocated to the mth UE. $g_{n,m}$ is the channel gain of the transmission link between the nth BS and the mth UE which is decided by the free space propagation model. M_n denotes the set of UE associated with the nth BS. $P_{n,r}$ denotes the transmission power and N_0 denotes the power density of the noise.

C. Problem Formulation

According to the service type of UEs, we have different optimization objectives for different UEs since they have different QoS requirements. If a UE has a GBR service, the utility of a UE is decided by the delay of the traffic, which is given by Eq. (1). If a UE has a non-GBR service, the utility of a UE is decided by the throughput of the UE, which is decided by the packet arrival and the channel capacity given by Eq. (3). To constrain the utility value between 0 and 1, we use the estimated maximum throughput and delay of UEs, B_{max} and D_{max}, for normalization. Furthermore, since we want to minimize the delay time rather than maximize it, we define the utility of GBR traffic as $1 - \frac{d_m}{D_{max}}$. Therefore, for the mth UE, the TS problem is formulated to maximize the utility:

$$\max_{\beta_{m,n}} \quad U_m, \tag{5}$$

$$s.t. \quad (1)-(2)$$

$$U_m = \begin{cases} \frac{b_m}{B_{max}}, & if \ \ T_m = 0 \\ 1 - \frac{d_m}{D_{max}}, & if \ \ T_m = 1 \end{cases} \tag{5a}$$

$$\alpha_{n,r,m} = \{0,1\} \forall n, r \tag{5b}$$

$$\sum_{m \in M_n} \alpha_{n,r,m} = 1 \quad \forall r \in R_n, n \in N \tag{5c}$$

$$M_n = \{m' | \forall \beta_{m',n} = 1\} \tag{5d}$$

$$\beta_{m,n} = \{0,1\}, \forall n \tag{5e}$$

$$\sum_{n \in N} \beta_{n,m} = 1 \tag{5f}$$

where Eq. (1)–(2) defines the variable d_m which used in the definition of the utility. Eq. (5a) indicates that the optimization goal of each UE is related to its traffic type T_m. $T_m = 0$ indicates the mth UE holds non-GBR traffic and the utility is related to its throughput b_m. $T_m = 1$ indicates the mth UE holds a GBR traffic, and its reward is related to its delay d_m. B_{max} and D_{max} are the estimated maximum throughput and delay of UEs and they are fixed values to normalize the utility of delay and throughput. $\beta_{m,n}$ is a binary indicator denoting whether the mth UE chooses the nth BS. Eq. (5b) indicates that a PRB can be assigned or unassigned to a user. Eq. (5c) indicates that each physical resource block of a BS can only be allocated to one UE at the same time. Eq. (5d) indicates that $\beta_{m,n}$ decides the attached list of BSs and Eq. (5f) indicates that each UE can only choose one BS at one time. Given the UE-centric structure of our proposed framework, it is worth noting that the problem formulation in Eq. (5) is defined for one single UE. According to this problem formulation, each UE makes local decisions to maximize its own utility.

As it can be observed in Eq. (5), the proposed problem contains uncertain or stochastic parameters, and the optimal solution may depend on the probability distribution of the parameters. In addition, in the TS scenario, the BS selection of a UE at a given moment affects not only the immediate QoS rewards but also the QoS rewards in the future. We want to focus more on the long-term rewards, and it is difficult to complete by optimization. Therefore, the formulated problem could not be directly solved, and an AI-based solution is proposed.

IV. Proposed TS Scheme

In this section, we present the proposed KT-FLUC method. First, we provide an overview of the algorithmic framework. Then, we describe the federated reinforcement learning algorithm, model compression method, and knowledge transfer technique we used in KT-FLUC, respectively.

A. Algorithm Framework

Fig. 2 shows the framework of the proposed UE-centric TS method. Firstly, we apply the growing and pruning method to decide the optimal network size for the UR intelligence. In particular, we first grow the NN size by adding neurons, then compress the NN size by pruning neurons. Given the appropriate NN size, federated learning and knowledge transfer are cooperatively performed within a KU-FLUC framework. This framework uses an attention-weighted federated learning algorithm to aggregate experience from distributed UEs and generates a global model. A weight parameter is calculated for each UE by an attention model according to the performance of each UE. Then, UEs submit the parameters of their local models to the RAN controller, and the parameters of the global model are updated according to the local parameters and attention weights. After that, the knowledge transfer technique is used to transfer the aggregated experience from the global model to UEs. It is implemented by defining an expert model for each UE. The expert is a copy of the aggregated global

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on October 05,2026 at 18:03:27 UTC from IEEE Xplore. Restrictions apply.

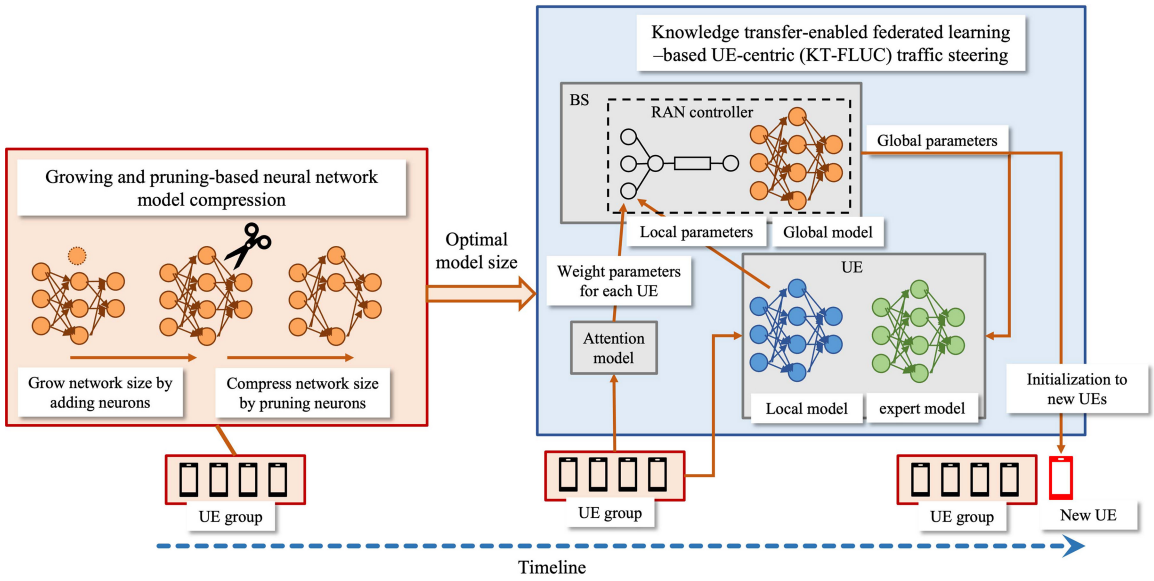


Fig. 2. The framework of proposed UE-centric TS method.

model, which runs inside each UE. In the local training process of each UE, only local models are trained, and expert models remain unchanged. Every time the global model is updated, the parameters of expert models will also be updated by parameter replication, and the action selection is processed inside each UE according to the outputs of expert models and local models.

In addition, for realistic dynamic environments, new UEs may join the network without prior knowledge of intelligent TS. Knowledge transfer will be performed between the global model and the newcomer for initialization. After that, the newcomer will perform local training and join the next round of federated learning.

This KT-FLUC framework uses federated learning to collect and aggregate experience from UEs with similar traffic types and QoS requirements. Knowledge transfer transfers the aggregated experience to existing UEs and newcomers. The expert model of knowledge transfer is obtained during federated learning, which is how these two techniques are connected [33]. The update of the global model by federated learning is non-real-time, while the TS decisions made by UEs are real-time. Under this framework, UEs may leave the system, but their experience will be kept for newcomers. The shared model is constantly updated with the distributed efforts of all the UEs in this network. Despite the high dynamics of the system, the knowledge learned by UEs consistently contributes to the optimization of a shared expert model. In turn, the expert model accelerates the local models' training and enhances system performance.

In the following, we first introduce the federated reinforcement learning-based UE-centric TS algorithm since it serves as the main scheme in this work. Then, we explain how to implement model compression for neural networks, and finally, transfer learning techniques are introduced.

B. Federated Reinforcement Learning Based UE-Centric TS

For UE-centric TS, each UE is treated as an independent agent, including a DQN for decision-making [34]. The Markov decision process (MDP) of UE-centric TS is defined as:

- *State:* The state of TS of each UE is composed of the received signal strength indicator (RSSI) from each BS, the number of queued packets, the delay of the last arrived packet, and its current attached BS. The RSSI information is accessible to UEs, and it can reveal the state information of the system, including the channel quality and the distance between the UE and BSs. The number of queued packets is related to the QoS of the UE. The delay of the last arrived packet, and the current attached BS indicates whether the UE needs to switch to a new BS. These indicators are important for TS decisions and can easily be observed and collected from the view of a single UE. The state at time t is given as:

$$S_{m,t} = \left\{ \Gamma_{n,t}, L_{m,t}^{queue}, d_{m,t}, \beta_{m,n,t} | n \in N \right\}, \quad (6)$$

  where Γ_n denotes the RSSI from the nth BS and L_m^{queue} denotes the current packet queue length.
- *Action:* After observing states from the environment, UE will make local TS decisions. Since TS is to select the best serving BS for each UE, the action of TS of each UE is to choose an optimal BS, which is given as:

$$A_{m,t} = \left\{ \beta_{m,n,t} | n \in N \right\}, \quad (7)$$

  where $\beta_{m,n}$ denotes whether the mth UE chooses the nth BS.
- *Reward:* The TS reward for each UE is related to their traffic type and the delay and throughput of the traffic,

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on October 05,2026 at 18:03:27 UTC from IEEE Xplore. Restrictions apply.

which can be given as:

$$R_{m,t} = \begin{cases} \frac{b_{m,t}}{B_{max}}, & if \ \ T_n = 0. \\ 1 - \frac{d_{m,t}}{D_{max}}, & if \ \ T_n = 1. \end{cases} \tag{8}$$

As it can be observed from the MDP definition, the definition of reward has a similar format to the utility, which is the optimization goal of the problem formulation. The action is similar to the control variable of the problem formulation. By doing this, we convert the problem defined in Eq. (5) into a MDP problem, which can then be solved with the DQN algorithm. The optimal gap of the solution is decided by the training effect of the DQN. In an ideal situation, the DQN is able to develop an optimal TS solution with full exploration and accurate prediction. However, it is very difficult for DQN to be trained to perfection in real cases. To reduce the optimal gap, in this work, we also use FL and transfer learning techniques to enhance the training effect of DQN. Also, to limit the number of handovers, the handover action will be selected only if the reward value differential is higher than a given threshold. In this way, the framework will avoid unnecessary handovers.

The core idea of DQN is to use a NN to predict the long-term reward, which is denoted as Q-values, of different actions under given states. The Q-values in each step can be updated according to the Bellman equation:

$$Q(s_{m,t}, a_{m,t}) = (1-\alpha) Q(s_{m,t}, a_{m,t}) + \alpha \Big(R_{m,t} + \max_a \gamma Q(s_{m,t+1}, a) \Big), \tag{9}$$

where $Q(s_{m,t}, a_{m,t})$ denotes the long-term reward of choosing action a under the observation of state s at time t of the mth UE. γ is the discount factor denoting the importance of distant future reward compared with instant reward, and α is the learning rate denoting the update speed of the NN. The local update of DQN at the mth UE can be formulated as:

$$\theta_{m,t+1} = \theta_{m,t} + \alpha \Big[R_{m,t} + \gamma \max_a Q(s_{m,t+1}, a; \theta_{m,t}) - Q(s_{m,t}, a_{m,t}; \theta_{m,t}) \Big] \nabla Q(s_{m,t}, a_{m,t}; \theta_{m,t}), \tag{10}$$

where $\theta_{m,t}$ denotes the parameters of the DQN at time t.

The federated learning scheme is shown in Fig. 3. Here, we adopted a grouped attention-weighted federated learning approach based on FedAvg, and multi-centered federated learning [35], [36], [37]. Considering the heterogeneity between different traffic types, we assume UEs with GBR traffic as the first group and UEs with non-GBR traffic as the second group and enable model federating and experience sharing among groups. Inside each group, an attention weight is calculated and assigned to each UE to represent the contribution of each UE to the global model and promote model aggregation.

The federated learning process inside each group can be concluded as three main steps. The first step is the local training and updates summarized in Algorithm 1. In this step, each UE observes local states, selects actions according to the Q-values generated by local DQN, and uses local experience

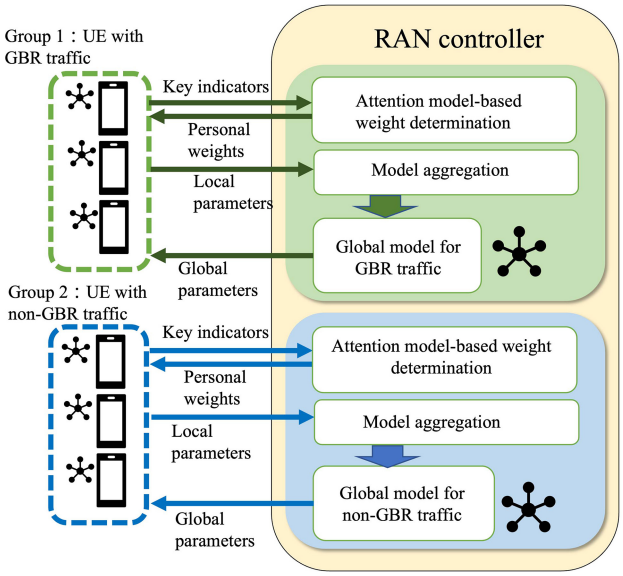


Fig. 3. Federated learning process.

Algorithm 1 Local Training at Each UE

1: Observe local state.
2: $S_{m,t} \leftarrow \{\Gamma_n, L_n^{queue}, d_m, \beta_{m,n} | n \in N\}$.
3: Get immediate reward $R_{m,t}$.
4: Record experience $\{S_{m,t+1}, S_{m,t}, a_{m,t}, R_{m,t}\}$.
5: Sample a minibatch from experience buffer
6: Update local Q network θ with $\nabla(L^{local}(\theta_{m,t}))$ via Equation (10)
7: Choose actions by $a_{m,t} \leftarrow \underset{a_{m,t}}{argmax} Q(S_{m,t}, a_{m,t}; \theta_{m,t})$.
8: Execute action $a_{m,t}$.

to train DQN by minimizing the local loss function. The local loss function is defined as the mean squared error, which can be given as:

$$Loss_m^{local}(\theta_{m,t}) = \sum_{\Psi_m} \Big(y_m - R_{m,t} - \gamma \max_a Q(s_{m,t+1}, a; \theta_{m,t}) \Big)^2, \tag{11}$$

where $Loss_m^{local}$ denotes the local loss function of the mth UE and Ψ_m denotes the memory buffer of the mth UE. y_m denotes the output of the local DQN.

The second step is attention weights determination. In each federating interval, all the UEs submit the parameters of their local DQN to the RAN controller to aggregate a global model. Weight parameters are calculated and assigned for each UE as preparation for aggregation. Since all the local models are trained separately and independently, they are trained at different degrees. To generate a more accurate and generalized global model, we expect well-trained models to contribute more to the global model. In contrast, barely trained models contribute less to the global model. Given this, three key indicators are used to evaluate how well the model has been trained, the average reward, training data size and the achievement rate.

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on October 05,2026 at 18:03:27 UTC from IEEE Xplore. Restrictions apply.

- The average reward of a UE is denoted as $\overline{R_m}$ and refers to the average value of system reward defined by Equation (8) of the local DQN during the interval of two federating processes. It can be seen as an intuitive representation of the DQN model performance.
- The training data size of a UE is denoted as $|\Psi_m|$ and refers to the amount of training data. We expect UEs with richer experience to account for more weight in federating.
- The achievable rate of a UE is denoted as Φ_m and refers to the frequency that the UE achieves a specific level of latency or throughput requirement. We add this indicator to $\overline{R_m}$ to prevent coincidental large rewards from giving a false representation of model performance.

After obtaining all the key indicators from the UE side, we use a SoftMax function-based attention mechanism to combine these indicators and decide a weight for each local model accordingly:

$$w_m = softmax\left(\frac{QK_m}{\sqrt{n_k}}\right), \tag{12}$$

where w_m denotes the weight of the mth UE. n_k is the dimension of indicators. K_m is the key indicators and can be formulated as $K_m = [\frac{\overline{R_m}}{\max\limits_m \overline{R_m}}, \frac{|\Psi_m|}{\max\limits_m |\Psi_m|}, \frac{\Phi_m}{\max\limits_m \Phi_m}]$. Q is the query and can be formulated as $Q = [1, 1, 1]$. By defining the key and the query, we expect UEs with indicators closer to the maximum values receive more attention. For instance, if a UE has a higher average reward value and a higher achievable rate, it is considered to be better trained. If it has a larger training data size, the training may considered to be more reliable. Therefore, we firstly pre-process the indicators by finding the maximum $\overline{R_m}$, $|\Psi_m|$ and Φ_m among all the UEs and perform normalization. The weight can then be calculated as:

$$\begin{aligned} w_m &= softmax\left(\frac{\overline{R_m}' + |\Psi_m|' + \Phi'_m}{\sqrt{n_k}}\right) \\ &= \frac{\exp\left(\frac{\overline{R_m}' + |\Psi_m|' + \Phi'_m}{\sqrt{n_k}}\right)}{\sum_{m' \in M} \exp\left(\frac{\overline{R'_{m'}}' + \left|\Psi'_{m'}\right|' + \Phi'_{m'}}{\sqrt{n_k}}\right)}, \end{aligned} \tag{13}$$

where $\overline{R_m}'$, $|\Psi_m|'$ and Φ'_m denotes the indicators after normalization.

Finally, the last step is model aggregation and model update. The parameter of global model can be obtained as the weighted sum of all the local models, which is given as:

$$\theta_{g,t+1} = (1 - \eta_1)\theta_{g,t} + \eta_1 w_m \sum_{m \in M} \theta_{m,t}, \tag{14}$$

where θ_g denotes the parameters of the global model. η_1 is a parameter denoting the update speed of global model parameters. θ_m denotes the parameters of the mth local model. In this way, the well-trained models will be given larger weights and contribute more to the global model. The barely trained models will be given smaller weights and contribute less to the global model.

Algorithm 2 Grouped Attention-Weighted Federated Reinforcement Learning-Based TS

Initialize: θ_{local} of each UE, S^{t-1} and a^t
while $TTI < T^{total}$ **do**
  for each UE **do**
    Do local training via Algorithm 1
  end for
  if $mod(TTI, FedInterval) == 0$ **then**
    for each UE **do**
      Submit θ_{local} to RAN controller.
      $\overline{R}_n \leftarrow \sum_{TTI \in FedInterval} R_{TTI} / FedInterval$
      $|\Psi_m| \leftarrow length\{buffer\} / T^{total}$
      $\phi_m \leftarrow \sum_{TTI \in FedInterval} Eligible_m / FedInterval$
      $w_n \leftarrow softmax(\overline{R}_n, |\Psi_m|, \phi_m)$
    end for
    Update global model according to Equation (14)
    for each UE **do**
      Update local models via Equation (15)
    end for
    $TTI \leftarrow TTI + 1$
  end if
end while

Then the local models are updated with the global model, which is given as:

$$\theta_{m,t+1} = (1 - \eta_2)\theta_{m,t} + \eta_2 \theta_{g,t}, \tag{15}$$

where η_2 is a parameter denoting the update speed of local model parameters during federating. The grouped attention-weighted federated reinforcement learning approach is summarized at Algorithm 2, where $Eligible_m$ is a binary indicator denoting whether the UE achieves a specific level of latency or throughput requirement and *FedInterval* denotes the time interval between two times of model aggregations in federated learning.

C. Growing and Pruning-Based Model Compression

This section will introduce the growing and pruning method for model compression. Based on biological neural theories, neurons compete with each other. The neurons with more substantial competitiveness are suitable for survival and growth while the neurons with weaker competitiveness will be eliminated [38], [39]. Therefore, the model compression can be divided into two phases, the growing phase and the pruning phase, shown in Fig. 4.

As shown in Fig. 4, a growing phase is first performed to estimate how many neurons will be enough to support the network functions. It is worth explaining that although UEs are resource-constrained, the growing process is designed before our real implementation and does not have to be run online on UE devices. So the constrained resources will not be a problem. During the growing phase, the number of neurons is added one by one by splitting the neuron with the strongest competitiveness into two. If no improvement is observed in the network performance as the model size increases, the NN is

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on October 05,2026 at 18:03:27 UTC from IEEE Xplore. Restrictions apply.

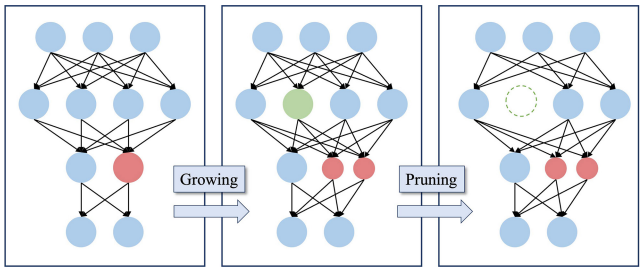


Fig. 4. Two phases of growing and pruning.

supposed to be large enough to support the network functions. Then the algorithm steps into the pruning process and the neurons are pruned one by one by cutting off the neuron with the weakest competitiveness until there are no more neurons to be cut off.

The competitiveness of a neuron is defined according to the average percentage of zeros (PoZ) by measuring the percentage of zero activations of a neuron after the rectified linear activation unit (ReLU) mapping [40], which is given as:

$$PoZ_i^j = \frac{\sum_z^Z \sum_l^L f\left(o_{l,j}^{(i)}(z) == 0\right)}{Z * L}, \tag{16}$$

where PoZ_i^j denotes the average PoZ of the jth neuron in the ith layer. L and Z denote the dimension of the neuron output and the number of validation samples. $o_{l,j}^{(i)}$ denotes the lth output of the jth neuron in the ith layer. $f(*)$ is a binary logic function, which can be given as:

$$f(*) = \begin{cases} 1, & if\ true. \\ 0, & if\ false. \end{cases} \tag{17}$$

Since the ReLu mapping will change all the negative inputs into zero, the average PoZ of Relu mapping can reflect the percentage of discarded and reserved neuron information. The neuron with the highest average PoZ is supposed to reserve the minor information and have the weakest competitiveness. In contrast, the neuron with the lowest average PoZ keeps the most information and have the most substantial competitiveness.

During the growing phase, when a neuron is split into two, the parameters of new neurons are initialized with the parameters of the old neuron. This can ensure that the split process will not influence the training process and decay the NN performance. The initialized weights of new neurons after splitting the jth neuron at the ith layer is given as:

$$w_{l,j}^{(i)'} = \Delta w_{l,j}^{(i)} \qquad \forall l \in N_{i-1}, \tag{18}$$

$$w_{l,j+1}^{(i)'} = (1-\Delta) w_{l,j}^{(i)} \qquad \forall l \in N_{i-1}, \tag{18a}$$

$$w_{j,k}^{(i+1)'} = w_{j+1,k}^{(i+1)'} = w_{j,k}^{(i+1)} \qquad \forall k \in N_{i+1},$$
$$\Delta \in (0,1) \tag{18b}$$

where $w_{l,j}^{(i)}$ denotes the weight of the lth dimension of the jth neuron at the ith layer, and $w_{l,j}^{(i)'}$ denotes the updated weight value after neuron split. N_i denotes the dimension of neurons at the ith layer and $w_{l,j}^{(i)'}$ denotes the updated weight after neuron split. Δ is a parameter for segmentation that can be chosen from zero to one. To make a proof that there is no decay in NN performance before and after the neuron split, we calculate the output of the jth neuron and the $(j+1)^{th}$ neuron at the ith layer with the initialized weight after the neuron split as follows:

$$o_j^{(i)'} = \sum_{l=1}^{N_{i-1}} w_{l,j}^{(i-1)'} o_l^{(i-1)} = \Delta \sum_{l=1}^{N_{i-1}} w_{l,j}^{(i-1)} o_l^{(i-1)} = \Delta o_j^{(i)}, \tag{19}$$

$$o_{j+1}^{(i)'} = \sum_{l=1}^{N_{i-1}} w_{l,j+1}^{(i-1)} o_l^{(i-1)} = (1-\Delta) \sum_{l=1}^{N_{i-1}} w_{l,j}^{(i-1)} o_l^{(i-1)}$$
$$= (1-\Delta) o_j^{(i)}, \tag{20}$$

Since the format of ReLu is given as follows:

$$ReLu(x) = x^+ = max(x, 0), \tag{21}$$

we can make inferences from ReLu mapping as follows:

$$\because\ ReLu((1-\Delta)o_j^{(i)}) = (1-\Delta) ReLu(o_j^{(i)})$$
$$ReLu(\Delta o_j^{(i)}) = \Delta ReLu(o_j^{(i)}) \tag{22}$$
$$\therefore ReLu((1-\Delta)x_j^{(i)}) + ReLu(\Delta x_j^{(i)})$$
$$= (1-\Delta) ReLu(o_j^{(i)}) + \Delta ReLu(o_j^{(i)})$$
$$= ReLu(x_j^{(i)}), \tag{23}$$

So, the output of any neurons at the $(i+1)^{th}$ layer can be given as:

$$\forall k \in N_{i+1},$$
$$o_k^{(i+1)'} = \sum_{J=1}^{N_i} w_{J,k}^{(i+1)'} ReLu(o_J^{(i)'})$$
$$= \sum_{J=1}^{j-1} w_{J,k}^{(i+1)'} ReLu(o_J^{(i)'}) + w_{j,k}^{(i+1)'} ReLu(o_j^{(i)'})$$
$$+ w_{j+1,k}^{(i+1)'} ReLu(o_{j+1}^{(i)'}) + \sum_{J=j+2}^{N_i} w_{J,k}^{(i+1)'} ReLu(o_J^{(i)'})$$
$$= \sum_{J=1}^{j-1} w_{J,k}^{(i+1)} ReLu(o_J^{(i)}) + w_{j,k}^{(i+1)} ReLu((1-\Delta)o_j^{(i)})$$
$$+ w_{j,k}^{(i+1)} ReLu(\Delta o_j^{(i)}) + \sum_{J=j+1}^{N_i} w_{J,k}^{(i+1)} ReLu(o_J^{(i)})$$
$$= \sum_{J=1}^{j-1} w_{J,k}^{(i+1)} ReLu(o_J^{(i)}) + w_{j,k}^{(i+1)} ReLu(o_j^{(i)})$$
$$+ \sum_{J=j+1}^{N_i} w_{J,k}^{(i+1)} ReLu(o_J^{(i)})$$
$$= o_k^{(i+1)}, \tag{24}$$

where it can be observed that the output of any neurons at the $(i+1)^{th}$ layer after neuron split is the same as the output before the neuron split. It proves that there is no performance decaying after the neuron split. The model compression procedure is summarized at Algorithm 3.

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on October 05,2026 at 18:03:27 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3 Model Compression for Hidden Neuron Number Determination

Initialize: $N_1, N_2 \leftarrow 2$, θ_m^{local}, S_m and A_m of each UE, $\overline{P_{T-1}} \leftarrow -\infty, N_{split} \leftarrow 0, \overline{P_T} \leftarrow 0$
while $mod(TTI, SplitInterval)! = 0$ **do**
  for each UE **do**
    Do local training via Algorithm 1
    $\overline{P_T} \leftarrow \overline{P_T} + R_0$
  end for
  $TTI \leftarrow TTI + 1$
end while
$TTI \leftarrow TTI + 1$
if $N_{split} \leq N_{required}$ **then**
  if $P_T < P_{T-1}$ **then**
    $N_{split} \leftarrow N_{split} + 1$
  else
    $N_{split} \leftarrow 0$
  end if
end if
Calculate PoZ_i^j in *SplitInterval* via Equation (16)
if $N_{split} \leq N_{required}$ **then**
  $j_i = \max_j PoZ_i^j \quad \forall i = 1, 2$
  $N_1 \leftarrow N_1 + 1$
  $N_2 \leftarrow N_2 + 2$
  Split the jth neuron at the $\forall i = 1, 2$ layer and initialize $w_{l,j}^{(i)'}$, $w_{l,j+1}^{(i)'}$ and $w_{l,j}^{(i+1)'}$ via Equation (18)
  $P_{T-1} = P_T$
  go to Line 2
else
  $i, j = \min_{i,j} PoZ_i^j$
  if $i == 1$ and $N_1 > 2$ **then**
    $N_1 \leftarrow N_1 - 1$
  else
    $N_2 \leftarrow N_2 - 1$
  end if
  Cut off the jth neuron at the ith layer and record N_1, N_2 and P_T
  if $N_1 > 2$ or $N_2 > 2$ **then**
    go to Line 2
  end if
end if

D. Q-Value-Based Knowledge Transfer

This section introduces the transfer learning technique. The framework of knowledge transfer is shown in Fig. 5. We define two models for each UE, an expert model and a local model. During the TS process, both the expert and local models of a UE observe the state from the environment and generate Q-vales for different actions. The expert model is initialized by the global model in the RAN controller through parameter replication. Although expert models share the same network parameters with the global model, they are different models because they are deployed at different parts of the network and play different roles. The expert model is deployed inside each UE. It takes the previously learned knowledge of the

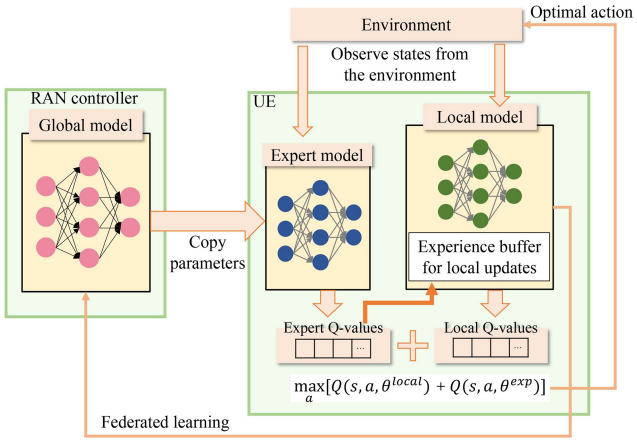


Fig. 5. Knowledge transfer process.

global model as expert knowledge and provides expert Q-values based on the global experience during local training process in federated learning. The local model provides local Q-values based on the local experience. The TS actions are chosen by maximizing the sum of expert and local Q-values. This process can be formulated as:

$$a = \underset{a}{argmax}\left(Q\left(s_{m,t}, A; \theta_{m,t}^{local}\right) + Q\left(s_{m,t}, A; \theta_{m,t}^{exp}\right)\right), \tag{25}$$

where θ^{local} denotes local model parameters and θ^{exp} denotes expert model parameters. In this way, the global experience from the expert model and the local experience from the local model are combined for decision-making.

When a new UE arrives at the system, the global model parameters are directly copied into the local and expert models for initialization. Next, during each federated interval, the actions are chosen as a combined effort of both the local and expert models. We use separated local models and expert models since the local model is trained and updated during local training, while the global model remains unchanged. When federated learning is performed, the global model is updated by federating all the local models. The updated global model is used to update the expert models at each UE side. Therefore, the global experience and local experience can be simultaneously learned but separately kept at each UE. Another advantage of using the proposed knowledge transfer scheme is that it can reduce the running time by performing global model aggregation and local training simultaneously. Conventionally, the local model update is based on feedback from the global model, so local training and model aggregation should be performed sequentially. But with the introduction of the expert model, feedback is given to the expert model instead of the local model. That means UEs do not need to wait for feedback from the global model and model aggregation can be run in parallel with local training. In this way, the running time of model aggregation is saved.

Since the global model is an aggregation of local models and expert models are transferred from the global model, the local models and expert models are supposed to converge to

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on October 05,2026 at 18:03:27 UTC from IEEE Xplore. Restrictions apply.

the same value. Therefore, we redefine the loss function of local training as:

$$Loss_m = \sum_{\Psi_m} \big(y_m + Q\big(s_{m,t}, a_{m,t}; \theta^{exp}_{m,t}\big) - 2R_{m,t} - \gamma \max_a \big(Q\big(s_{m,t+1}, a; \theta^{local}_{m,t}\big) + Q\big(s_{m,t+1}, a; \theta^{exp}_{m,t}\big)\big)^2. \tag{26}$$

Since the expert Q model is not updated during the local training, $Q(s_{m,t+1}, a; \theta^{exp}_{m,t})$ and $Q(s^t_m, a^t_m; \theta^{exp}_{m,t})$ can be seen as constants. So the update of local models at each UE can be formulated as:

$$\theta_{m,t+1} = \theta_{m,t} + \alpha\big[2R_{m,t} - Q\big(s_{m,t}, a_{m,t}; \theta^{exp}_{m,t}\big) - \gamma \max_a \big(Q\big(s_{m,t+1}, a; \theta^{local}_{m,t}\big) + Q\big(s_{m,t+1}, a; \theta^{exp}_{m,t}\big)\big) - Q\big(s_{m,t}, a_{m,t}; \theta^{local}_{m,t}\big)\big] \nabla Q\big(s_{m,t}, a_{m,t}; \theta^{local}_{m,t}\big). \tag{27}$$

With this definition, the sum of expert Q-values and local Q-values is supposed to converge to twice the long-term accumulated reward. On the other hand, expert Q-values and local Q-values will converge to a similar value according to the federated learning process. So the final output is expected to converge to the long-term accumulated reward of given actions and states. In this way, expert models can provide a lead for UEs according to the previous experience of other UEs and UEs can make personalized modifications based on the suggestions of experts. It is worth pointing out that the expert models do not need to be trained on the UE side. It is only used for model inference during the local training, which only requires minimum computation.

As a combination of the aforementioned grouped attention-weighted federated learning and transfer learning, the complete version of the proposed federated reinforcement learning-based UE-centric TS is summarized at Algorithm 4.

Algorithm 4 KT-FLUC-Based TS

Initialize: $\theta_{local}, \theta^{exp}_{local}$ of each UE, S^{t-1} and a^t
while $TTI < T^{total}$ **do**
  $TTI \leftarrow TTI + 1$
  for each UE **do**
    Observe local state.
    $S \leftarrow \{\Gamma_k, L^{queue}_k, delay_n, k_n | k \in K\}$.
    Get immediate reward R.
    Record experience $\{S_{m,t+1}, S_{m,t}, a_{m,t}, R_{m,t}\}$.
    Sample a minibatch from experience buffer
    Update local Q network θ with
    $\nabla(L_{exp}(\theta_{local}, \theta_{exp}))$ via equation (26).
    Choose actions via equation (25).
    Execute action A.
  end for
  if $mod(TTI, FedInterval) == 0$ **then**
    for each UE **do**
      Submit θ_{local} to RAN controller.
      $\overline{R}_m \leftarrow \sum_{TTI \in FedInterval} R_{TTI}/FedInterval$
      $|MB_m| \leftarrow length\{buffer\}/T^{total}$
      $\Phi_m \sum_{TTI \in FedInterval} h_m/FedInterval$
      $w_m \leftarrow softmax(\overline{R}_m, |MB_m|, \Phi_m)$
    end for
    $\theta_{global} \leftarrow (1-\eta_1)\theta_{local} + \eta_1 w_m \sum_{m \in M} \theta_m/|M|$
    for each UE **do**
      $\theta^{exp}_{local} \leftarrow \theta_{global}$
    end for
  end if
  if A new UE enters the network. **then**
    $\theta_{local} \leftarrow \theta_{global}$
    $\theta^{exp}_{local} \leftarrow \theta_{global}$
  end if
end while

E. Baselines

In this paper, we proposed four baseline algorithms. Firstly, we compared our proposed KT-FLUC algorithm with a centralized cell-centric DRL-based TS algorithm (CL), which is implemented based on [24]. In this algorithm, we only use a single agent at the RAN controller, collecting observations from distributed UEs and making a centralized decision. The action is a joint TS decision of all the UEs, which will be broken into independent TS actions for each UE, and the model will be trained based on the average reward of all the UEs. The critical steps of the CL algorithm are summarized at Algorithm 6.

Then, we compared our proposed KT-FLUC algorithm with three other UE-centric algorithms. The first one is a distributed independent DRL-based TS algorithm (DIL), which is implemented based on the idea of [41]. In this baseline, each UE acts as an independent agent, making decisions without information exchanging or experience sharing. The activities of each UE in DIL are summarized at Algorithm 1.

The third baseline is a federated DRL-based TS algorithm (FL). The federated learning algorithm used in this baseline is borrowing ideas from [35]. Compared with DIL, the federated learning steps are added, while compared with the proposed algorithm, the knowledge transfer part is removed. The FL algorithm is summarized at Algorithm 2. The last baseline is a federated DRL-based TS algorithm with direct initialization (FLI). Direct initialization means that the global model parameters are directly copied to the local model every time a new UE enters the system. The FLI algorithm is summarized at Algorithm 5.

F. Computational Complexity Analysis

In this section, we analyze the computational complexity of the proposed KT-FLUC framework. For each UE, the complexity of the DQN model deployed on the UE is dominated by the training and updating of NN, consisting of updating the weights of neurons and the bias. So the computational complexity is proportional to the number of NN layers, the dimension of neurons and the number of neurons on each layer, which can be given as $O(\sum_{i=1}^{I} N_{i-1}N_i)$, where N_i denotes the NN size at the ith layer. In addition, the knowledge transfer process also contributes to the complexity. The computation complexity of summing local and expert Q-values is proportional to the size of action space, which is given as $O(|A|)$. A denotes the action set of the UE.

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on October 05,2026 at 18:03:27 UTC from IEEE Xplore. Restrictions apply.

Algorithm 5 Federated DRL Based TS With Direct Initialization

Initialize: θ_{local} of each UE, S_{t-1} and a
while $TTI < T^{total}$ **do**
  $TTI \leftarrow TTI + 1$
  for each UE **do**
    Do local training via Algorithm 1
  end for
  if $mod(TTI, FedInterval) == 0$ **then**
    for each UE **do**
      Submit θ_{local} to RAN controller.
      $\overline{R}_m \leftarrow \sum_{TTI \in FedInterval} R_{TTI}/FedInterval$
      $|MB_m| \leftarrow length\{buffer\}/T^{total}$
      $\Phi_m \sum_{TTI \in FedInterval} h_m/FedInterval$
      $w_m \leftarrow softmax(\overline{R}_m, |MB_m|, \Phi_m)$
    end for
    $\theta_{global} \leftarrow (1-\eta_1)\theta_{local} + \eta_1 w_m \sum_{m \in M} \theta_n/|M|$
    for each UE **do**
      $\theta_{local} \leftarrow (1-\eta_2)\theta_{local} + \eta_2 \theta_{global}$
    end for
  end if
  if A new UE enters the network. **then**
    $\theta_{local} \leftarrow \theta_{global}$
  end if
end while

Algorithm 6 Cell-Centric DRL-Based TS

Initialize: θ_{local} of each UE, S_p and a
while $TTI < T^{total}$ **do**
  $TTI \leftarrow TTI + 1$
  for each UE **do**
    Observe local state.
    $S_m^t \leftarrow \{\Gamma_n, L_n^{queue}, d_m, \beta_{m,n} | n \in N\}$.
    Get immediate reward $R_m^t R$.
  end for
  Collect local states and get cell-based state S_c^t.
  Add rewards from all the UEs and get cell-based reward R_c^t.
  Record experience $\{S_c^{t-1}, S_c^t, a_c^t, R_c^t\}$.
  Sample a minibatch from experience buffer
  Update Q network θ
  Choose actions by $a_c^t \leftarrow \underset{a_c^t}{argmax}\, Q(S, a_c^t; \theta)$.
  Separate the cell-based action a_c^t into actions for each cell a_m^t.
  Execute action a_m^t.
end while

For the RAN controller, the complexity is dominated by the federated learning process. The computation complexity of the attention weight calculation can be formulated as $O(M)$, where M denotes the number of UEs. The complexity of global update is given as $O(M \sum_{i=1}^{I} N_{i-1} N_i)$. In summary, these analyses show that the computation complexity of our KT-FLUC framework is proportional to the NN size and the UE number.

In summary, these analyses show that the computational complexity of local training is proportional to the number of neurons. In practical implementation, the UE usually has limited memory and computation capacity, while it will take them significant processing of network parameters to perform applications deployed on to them [8]. The growth of NN will have a critical impact on energy, power, and timing. So it is necessary to limit the NN size. In this paper, we have kept the number of neurons as low as possible through neural network compression to save the computation cost of local training. The complexity of global updates is proportional to the number of UEs and the number of neurons. Although the complexity will increase when the number of UE increases, the global server is usually assumed to be powerful enough to deal with the high computational complexity. On the other hand, the global model aggregation is a non-real-time process, so it will not be computationally demanding. Therefore, the system has good scalability and can still work well when there are a large number of UEs.

V. Simulation Settings and Results

A. Simulation Settings

We consider a scenario with $N = 5$ BSs, an LTE MBS in the center and 4 5G SBSs in the surrounding. We consider a dynamic system and UEs will arrive and depart during the simulation. 40% UEs have GBR service and 60% UEs have non-GBR service. The simulation is repeated five times using MATLAB, and the average outcomes are presented along with a 95% confidence interval. The B_{max} and D_{max} we use during the simulation are 10 ms and 10 Mbps. The update speed of the global model η_1 is 0.5. The update speed of the local model η_2 is 1. The split interval we use during model compression is 150 TTIs. The $N_{required}$ is set as 5. During the simulation, we change the average number of UEs from $M = 25$ to $M = 65$ to see how the proposed algorithm performs under different UE numbers and traffic loads. Depending on the dimensions of the state space and action space, the number of hidden neurons is initially set to 32 and 64 for most UEs. Then we perform growth and pruning for one single UE and start with two hidden neurons in each layer. Other simulation details about network environment settings and deep learning settings are shown in Table I.

B. Performance Analysis Between UE-Centric TS and Cell-Centric TS

This section will compare the proposed KT-FLUC with cell-centric baseline CL and DIL.

Fig. 6 shows the convergence curves of DIL, CL and proposed KT-FLUC algorithms when the number of UE is 45. It can be observed that the CL algorithm can achieve a higher system reward than the DIL algorithm, but a lower reward than our proposed KT-FLUC algorithm. This demonstrates that the cell-based algorithm has an advantage in system performance compared with simple UE-centric algorithms because the cell-centric algorithms hold more comprehensive information and can coordinate between UEs.

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on October 05,2026 at 18:03:27 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SIMULATION SETTINGS

Network environment settings	UE traffic settings
Bandwidth: 20 MHz	**GBR traffic**
Subcarriers of each RB: 12	QCI: 2
Tx/Rx antenna gain: 15 dB	Proportion: 40%
Number of resource blocks: 100	Data length: 50 KB
Propagation model: 128.1 + 37.6log(Distance)	Packet size: 1600 Bytes
Log-Normal shadowing: 8 dB	Packet arrival mode: Constant bit rate data
Number of UEs: 25/35/45/55/65	Inter-arrival time between packets: 3 ms
Number of BSs: 5.	**Non-GBR traffic**
MBS settings	QCI: 6
Radio access technology: LTE	Proportion: 60%
Carrier frequency: 0.8 GHz	Data length: 250 KB
Transmission power: 40 W	Packet size: 3200 Bytes
Number of MBS: 1	Packet arrival mode: Constant bit rate data
Bandwidth: 10 MHz	Inter-arrival time between packets: 3 ms
Channel model: 3GPP urban macro	**Deep learning settings**
SBS settings	Initial learning Rate: 0.001
Radio access technology: 5G NR	Discount factor: 0.5
Carrier frequency: 3.5 GHz	Epsilon value: 0.05
Transmission power: 20 W	Experience pool: 200
Number of MBS: 4	Batch size: 64
Bandwidth: 20 MHz	Hidden layer: 2
Channel model: 3GPP urban micro	Federated interval: 30 TTIs

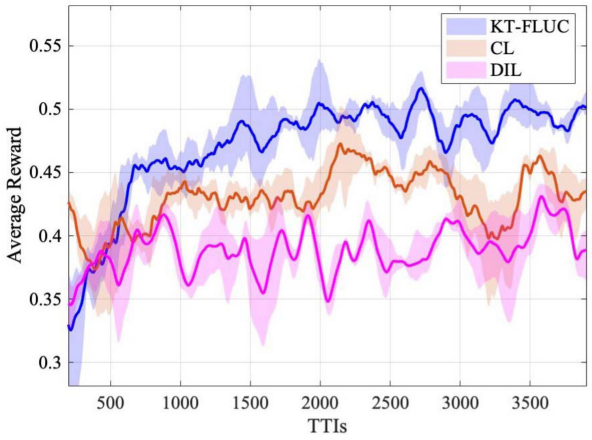
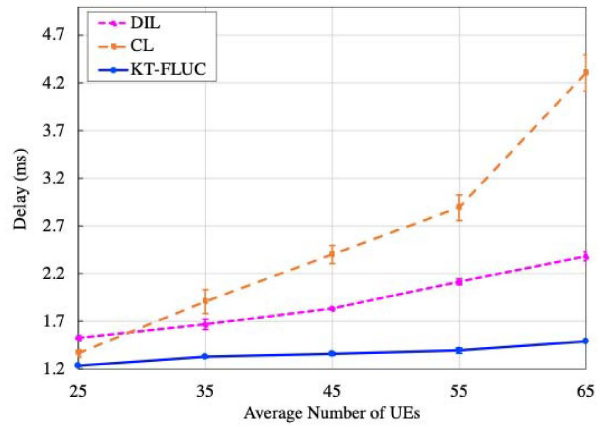
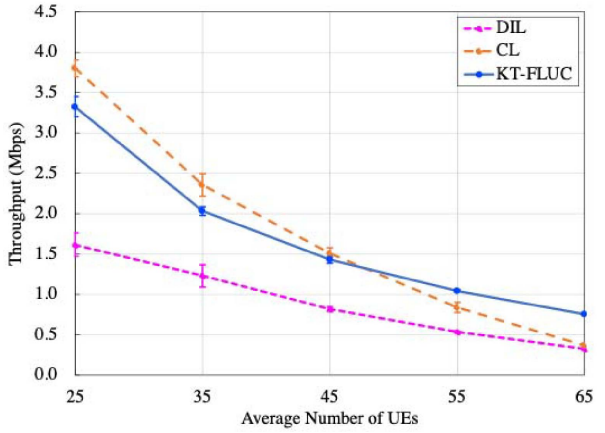


Fig. 6. The convergence performance of system reward.



(a) The average GBR traffic delay with different UE numbers.



(b) The average non-GBR traffic throughput with different UE numbers.

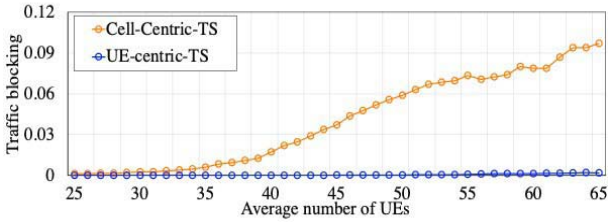
Fig. 7. The network performance of KT-FLUC, CL and DIL. (a) The average GBR traffic delay. (b) The average non-GBR traffic throughput.

In Fig. 7, the average delay of GBR traffic and throughput of non-GBR traffic under different numbers of UEs are shown. As the number of UE increases, the competition for resources among UEs becomes more intense, so the average delay will increase, and the average throughput will decrease. It can be observed that the CL algorithm performs better when there are fewer UEs. In contrast, UE-centric algorithms are influenced less by the increase in the number of UEs. This is because fewer UEs are easier to coordinate, and there is less information for the centralized model to deal with. When the average UE number is 25, the CL algorithm can achieve 14% higher throughput compared with the proposed KT-FLUC algorithm. However when the average UE number is 65, the proposed KT-FLUC algorithm can achieve 52% higher throughput than the CL algorithm. It can be observed that the delay of the CL algorithm is the highest among the three algorithms. This is because the cell-based algorithm tends to focus more on the overall performance of a cell rather than on the individual performance of UEs. Since the proportion of non-GBR traffic is much higher than GBR traffic, the CL algorithm may make decisions that are more suitable for non-GBR traffic at the expense of the benefits of GBR traffic to guarantee the overall reward. In comparison, in UE-centric algorithms, each UE fights for its own rewards, so it will choose the most appropriate actions for itself and makes the delay lower. When the average UE number is 65, the proposed KT-FLUC algorithm can achieve a 65% lower delay than the CL algorithm.

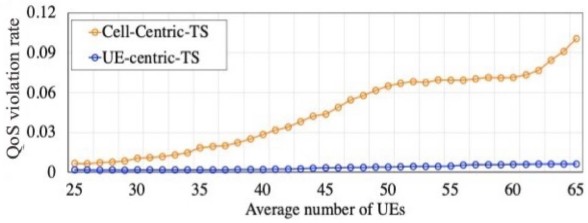
To further compare the cell-centric TS and UE-centric TS, we define three indicators to measure the UE's satisfaction with the TS service, traffic blocking, QoS violation rate and the transmission cycle. Since GBR traffic has a higher priority than non-GBR traffic, non-GBR traffic may be blocked when two kinds of traffic request transmission simultaneously. The traffic blocking represents the probability of GBR traffic being blocked. QoS violation rate represents the probability that the delay of GBR traffic exceeds the QoS requirements. And the last indicator, the transmission cycle, represents the average time for UEs to finish the transmission of a given file.

It can be observed from Fig. 8 that the UE-centric TS can achieve a lower traffic blocking and lower QoS violation rate compared with cell-centric TS algorithms. This verifies

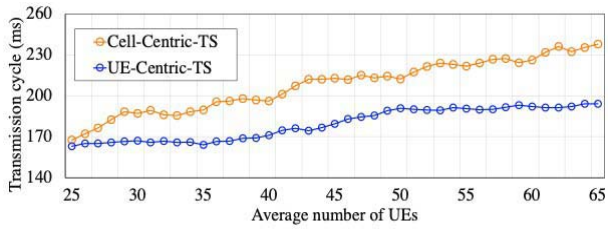
Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on October 05,2026 at 18:03:27 UTC from IEEE Xplore. Restrictions apply.



(a) The average traffic blocking of UEs of cell-centric TS and UE-centric TS.



(b) The average QoS violation rate of UEs of cell-centric TS and UE-centric TS.



(c) The average transmission cycle of UEs of cell-centric TS and UE-centric TS.

Fig. 8. The indicators of UE satisfaction as the number of UEs increases. (a) The average transmission cycle. (b) The average QoS violation rate. (c) The average transmission cycle.

that the UE-centric TS algorithm can respond more promptly to environmental changes. UE-based performance can give a more intuitive illustration of the UE's satisfaction than cell-based performance. The transmission cycle of UE-centric TS is also smaller than cell-centric TS, which means the traffic is transmitted faster in a UE-centric framework with more rational TS strategies.

C. Performance Analysis Between Four UE-Centric Frameworks

In this section, we compare the simulation results from DIL, FL, FLI and proposed KT-FLUC algorithms to analyze the contribution of transfer learning and federated learning techniques and to verify that the combination of two algorithms can create synergy.

Fig. 9 shows the convergence curves of DIL, FL, FLI and proposed KT-FLUC algorithms when the number of UE is 45. It can be observed that the DIL algorithm achieves the lowest system reward among the four algorithms. Furthermore, the DIL curves are oscillating and unstable, with large fluctuation intervals. In comparison, the FL algorithms can achieve higher system reward, and the FL curves are smoother. This is because in the DIL algorithm, there is no experience sharing between UEs and some UEs need to spend more time exploring and trying non-optimal actions. UEs may choose

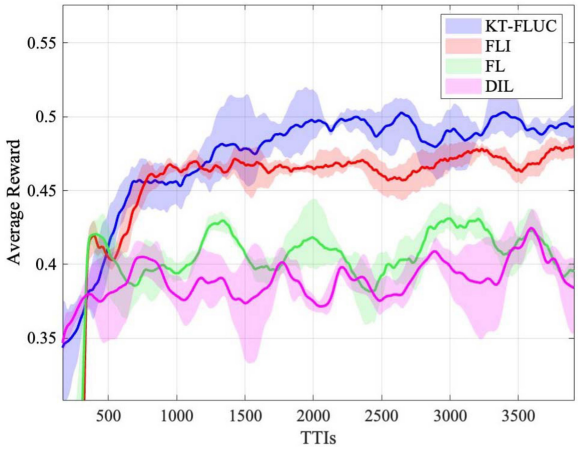


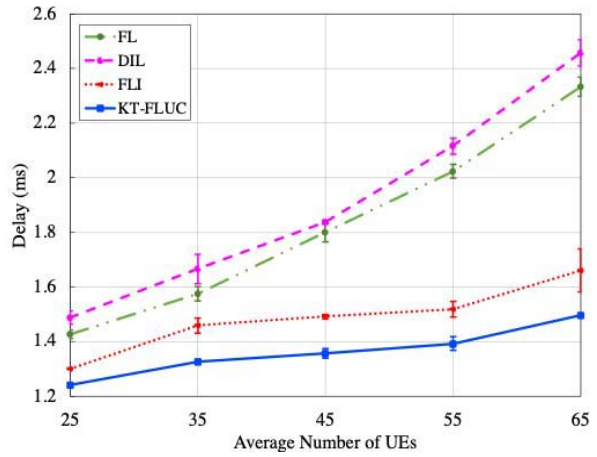
Fig. 9. The performance of system reward.

some bad actions during the exploration process, thus causing large fluctuations in the network performance and the reward value. Compared with the FL algorithm, the FLI algorithm can achieve higher system reward. This is because the parameter initialization from the expert model to newcomers can help them start faster. Also, newcomers may contribute lousy experience to the global model which will interfere with the experience sharing if they fail to learn practical experience before the federated learning process. The initialization can prevent users from contributing lousy experience, so it will help enhance the system performance. This can demonstrate that in addition to the regular role in federated learning, the global model can also play an additional role as an expert model for transfer learning. Finally, it can be observed that our proposed KT-FLUC algorithm can achieve the highest system reward, lowest delay, and highest throughput. This verifies that the proposed knowledge transfer algorithm performs better than copying parameters directly. The combination of federated learning and knowledge transfer can further improve system performance compared with federated learning.

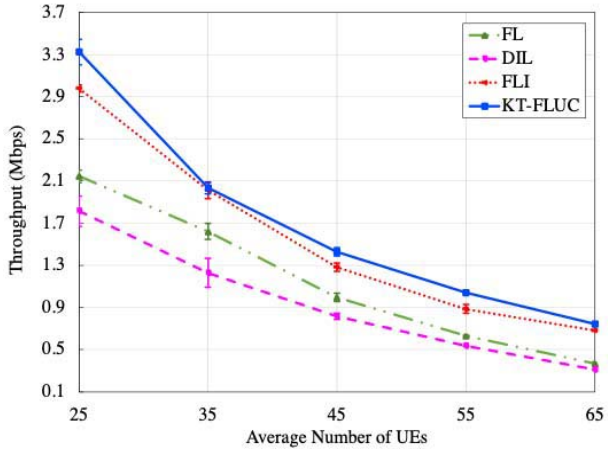
In Fig. 10, the average delay of GBR traffic and throughput of non-GBR traffic under different numbers of UEs are shown. With FLI and proposed KT-FLUC algorithms, there is no significant change in delay while the throughput will decrease when the number of UEs increases. This is because GBR traffic has a higher priority than non-GBR traffic, demonstrating that the traffic has been directed more effectively. In summary, the proposed KT-FLUC algorithm can consistently achieve the lowest packet delay and the highest throughput with different UE numbers. When the average UE number is 45, the proposed KT-FLUC algorithm can achieve 24% lower delay and 43% higher throughput compared with DIL algorithm. Also, with the proposed knowledge transfer scheme, it can achieve 10% lower delay and 10% higher throughput compared with FLI algorithm.

Table II further compares the running time of the KT-FLUC algorithm and the FLI algorithm. It shows the running time of different modules of the framework over 500 TTIs. In this table, the other time includes all other processes during the simulation, the PRB scheduling, and the data

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on October 05,2026 at 18:03:27 UTC from IEEE Xplore. Restrictions apply.



(a) The average GBR traffic delay with different UE numbers.



(b) The average non-GBR traffic throughput with different UE numbers.

Fig. 10. The network performance of KT-FLUC, FLI, FL and DIL. (a) The average GBR traffic delay. (b) The average non-GBR traffic throughput.

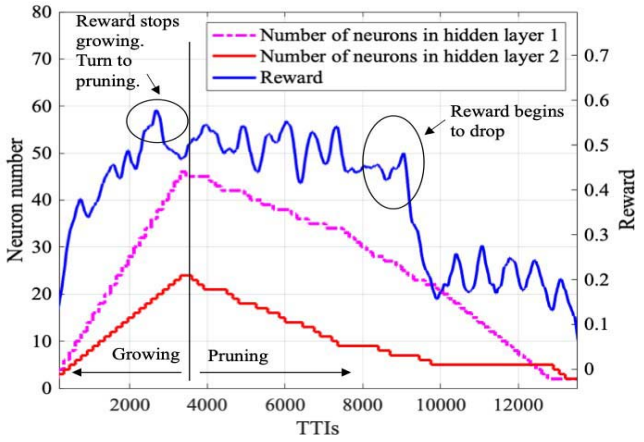
TABLE II
RUNNING TIME OF KT-FLUC AND FLI

	KT-FLUC	FLI
Federated learning time (s)	25.0549	28.6939
Knowledge transfer time (s)	0.0291	0.0243
Other time (s)	1.9063	1.8299
Total time (s)	26.9903	30.5481

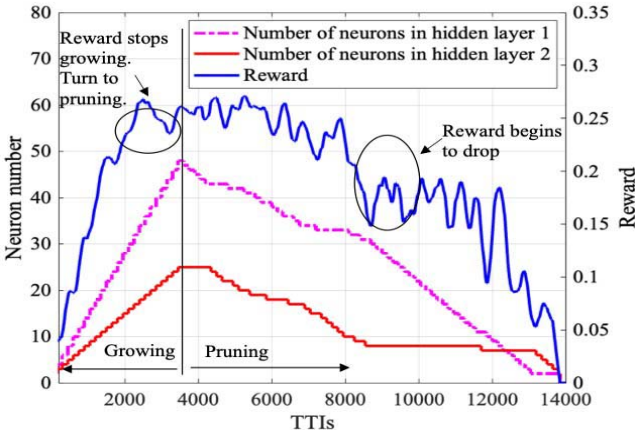
transmission. It can be observed that the knowledge transfer time of FLI is larger than KT-FLUC since the expert model will cause additional computational workload. However, the parallel running method used by KT-FLUC can efficiently reduce the federated learning time and total running time of the framework. Although extra calculation steps are introduced in KT-FLUC, it can still reduce the total running by 10%.

D. Performance Analysis of Model Compression

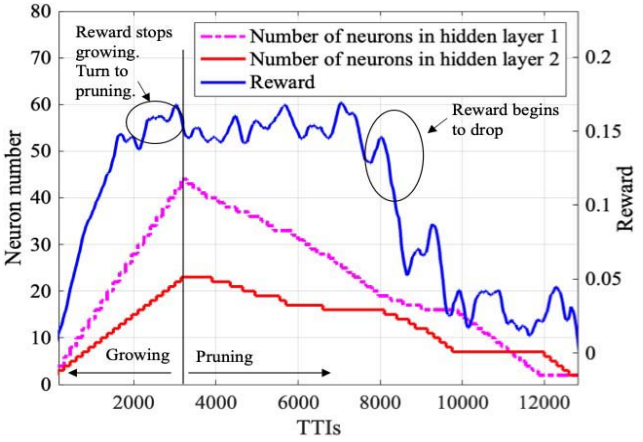
In this section, we analyze the simulation results of the pre-simulation for model compression. We assume the neuron number of the second layer is half of the first layer in the growing phase since the dimension of output layer is much smaller than the dimension of the input layer and 2:1 is a very



(a) The average number of UE in the system is 25



(b) The average number of UE in the system is 45



(c) The average number of UE in the system is 65

Fig. 11. The variation of neuron numbers in hidden layers and system reward during the growing and pruning simulation process under different UE numbers.

common assignment of neurons among two hidden layers. The variation of the number of neurons in the hidden layer and the system reward during the pre-simulation is shown in Fig. 11. The numbers of UEs in Fig. 11 (a), Fig. 11 (b) and Fig. 11 (c) are respectively 25, 45 and 65.

We take Fig. 11 (a) as an example to further illustrate the growing and pruning process for 32 UEs. In the beginning, the system is in a growing phase, and the neuron number

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on October 05,2026 at 18:03:27 UTC from IEEE Xplore. Restrictions apply.

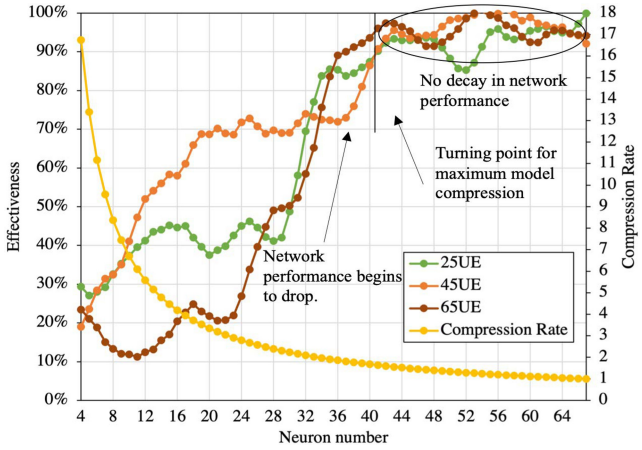


Fig. 12. Effectiveness and compression rate of different hidden neurons under different UE numbers.

of hidden layers increases with time. As a joint result of the response to increasing model size and model training, the system reward is also growing. Then during the 1800 - 2700 TTIs, a stabilization of the increase in reward is observed which is marked by an ellipse and arrow in the figure. The stabilization indicates that model training has begun to converge, and the current model size is large enough to support the TS application. So the simulation steps into the pruning process and the neuron numbers of both layers begin to decrease. Fig. 11 (b) and Fig. 11 (c) show similar processes. The neuron number increases during the growing phase the reward stabilizes and then decreases during the pruning phase.

At the beginning of the pruning process, the reward keeps stable as neurons with weaker competitiveness are pruned. This trend validates the theory that less competitive neurons have less influence on the output and demonstrates the possibility of reducing the number of hidden neurons without affecting the system performance to realize model compression. After about 8000 TTIs, there is a noticeable drop in the reward, meaning the corresponding model size is not large enough to support the given network function. The system performance will be influenced if we continuously compress the system model by pruning neurons. Therefore, by observing the reward during the pruning process, we can find a threshold as the maximum compression achievable without compromising system performance.

The further determine the compression threshold value, we defined an indicator called effectiveness by counting the average reward under a given model size and normalizing it with the maximum reward. The effectiveness and model compression rate under different model sizes is shown in Fig. 12. It can be observed that when the neuron number is larger than 40, few decays can be observed in the system performance of all three scenarios with effectiveness above 90%. On the other hand, when the neuron number is less than 40, the effectiveness of models has declined significantly to varying degrees. So we conclude that 40 neurons is the threshold value for the compressed model size and the corresponding compression rate, 1.5, is the maximum compression rate achievable without compromising performance. In this situation, there will be 80 network parameters need to be submitted during federated learning, and if there are 50 UEs, the size of information to be exchanged is about 8 KB. Considering that at the initial pruning phase, the neurons of two hidden layers are evenly pruned, we keep the two-fold relationship of neuron numbers of two hidden layers and use 14 and 28 neurons at each layer as our model size for following TS simulations.

VI. Conclusion

Machine learning is a promising solution for optimizing wireless networks. In this paper, we propose a federated transfer reinforcement learning-enabled UE-centric TS framework with knowledge transfer and model compression. The core idea is to use federated learning to aggregate decentralized local models, generate a robust global model, and use knowledge transfer to initialize newcomers. The simulation results show that the proposed framework achieves better convergence, stability, higher system throughput and lower delay compared with independent UE-centric TS and cell-centric TS. Considering these benefits, our proposed framework is not limited to TS. It can also be applied to other distributed wireless applications, such as edge caching and sleep control. In the future, we plan to extend UE-centric wireless networks to other applications.

References

[1] M. Elsayed and M. Erol-Kantarci, "AI-enabled future wireless networks: Challenges, opportunities, and open issues," *IEEE Veh. Technol. Mag.*, vol. 14, no. 3, pp. 70–77, Sep. 2019.

[2] C. Gijón, M. Toril, S. Luna-Ramírez, and M. L. Marí-Altozano, "A data-driven traffic steering algorithm for optimizing user experience in multi-tier LTE networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 10, pp. 9414–9424, Oct. 2019.

[3] S. Chandrashekar, A. Maeder, C. Sartori, T. Höhne, B. Vejlgaard, and D. Chandramouli, "5G multi-RAT multi-connectivity architecture," in *Proc. IEEE Int. Conf. Commun. Workshops (ICC)*, 2016, pp. 180–186.

[4] C. Zhang, M. Dong, and K. Ota, "Heterogeneous mobile networking for lightweight UAV assisted emergency communication," *IEEE Trans. Green Commun. Netw.*, vol. 5, no. 3, pp. 1345–1356, Sep. 2021.

[5] Y. Xu, W. Xu, Z. Wang, J. Lin, and S. Cui, "Load balancing for ultradense networks: A deep reinforcement learning-based approach," *IEEE Internet Things J.*, vol. 6, no. 6, pp. 9399–9412, Dec. 2019.

[6] F. Boccardi, R. W. Heath, A. Lozano, T. L. Marzetta, and P. Popovski, "Five disruptive technology directions for 5G," *IEEE Commun. Mag.*, vol. 52, no. 2, pp. 74–80, Feb. 2014.

[7] X. Wang, J. Li, L. Wang, C. Yang, and Z. Han, "Intelligent user-centric network selection: A model-driven reinforcement learning framework," *IEEE Access*, vol. 7, pp. 21645–21661, 2019.

[8] T. Mohsenin, I. Partin-Vaisband, H. Homayoun, J.-S. Seo, and X. Zhang, "Guest editorial cross-layer designs, methodologies, and systems to enable micro AI for on-device intelligence," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 11, no. 4, pp. 527–531, Dec. 2021.

[9] A. Mukherji, V. Srinivasan, and E. Welbourne, "Adding intelligence to your mobile device via on-device sequential pattern mining," in *Proc. ACM Int. Joint Conf. Pervasive Ubiquitous Comput. Adjunct Publ.*, New York, NY, USA, 2014, pp. 1005–1014.

[10] J. Konečný, B. McMahan, and D. Ramage, "Federated optimization: Distributed optimization beyond the datacenter," 2015, *arXiv:1511.03575*. [Online]. Available: http://arxiv.org/abs/1511.03575

[11] H. Flores, P. Nurmi, and P. Hui, "AI on the move: From on-device to on-multi-device," in *Proc. Pervasive Comput. Commun. Workshops*, 2019, pp. 310–315.

[12] M. Elsayed and M. Erol-Kantarci, "AI-enabled radio resource allocation in 5G for URLLC and eMBB users," in *Proc. IEEE 2nd 5G World Forum (5GWF)*, 2019, pp. 590–595.

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on October 05,2026 at 18:03:27 UTC from IEEE Xplore. Restrictions apply.

[13] H. Zhang, H. Zhou, and M. Erol-Kantarci,"Federated deep reinforcement learning for resource allocation in O-RAN slicing," in *Proc. IEEE Glob. Commun. Conf.*, 2022, pp. 958–963.
[14] R. Joda, T. Pamuklu, P. E. Iturria-Rivera, and M. Erol-Kantarci, "Deep reinforcement learning-based joint user association and CU-DU placement in O-RAN," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 4, pp. 4097–4110, Dec. 2022.
[15] C. Zhang, M. Dong, and K. Ota, "Fine-grained management in 5G: DQL based intelligent resource allocation for network function virtualization in C-RAN," *IEEE Trans. Cogn. Commun. Netw.*, vol. 6, no. 2, pp. 428–435, Jun. 2020.
[16] C. Zhang, M. Dong, and K. Ota, "Vehicular multi-slice optimization in 5G: Dynamic preference policy using reinforcement learning," in *Proc. GLOBECOM IEEE Global Commun. Conf.*, 2020, pp. 1–6.
[17] Y. Dantas et al., "Beam selection for energy-efficient mmWave network using advantage actor critic learning," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2023, pp. 5285–5290.
[18] H. Zhou, M. Erol-Kantarci, and H. Vincent Poor, "Learning from peers: Transfer reinforcement learning for joint radio and cache resource allocation in 5G RAN slicing," *IEEE Trans. Cogn.e Commun. Netw.*, vol. 8, no. 4, pp. 1925–1941, Dec. 2022.
[19] G. Zhu, D. Liu, Y. Du, C. You, J. Zhang, and K. Huang, "Toward an intelligent edge: Wireless communication meets machine learning," *IEEE Commun. Mag.*, vol. 58, no. 1, pp. 19–25, Jan. 2020.
[20] G. Ding, J. Yuan, J. Bao, and G. Yu, "LSTM-based active user number estimation and prediction for cellular systems," *IEEE Wireless Commun. Lett.*, vol. 9, no. 8, pp. 1258–1262, Aug. 2020.
[21] S. Singhand and J. G. Andrews, "Joint resource partitioning and offloading in heterogeneous cellular networks," *IEEE Trans. Wireless Commun.*, vol. 13, no. 2, pp. 888–901, Feb. 2014.
[22] M. Dryjanski, L. Kulacz, and A. Kliks, "Toward modular and flexible open RAN implementations in 6G networks: Traffic steering use case and O-RAN xApps," *Sensors*, vol. 21, no. 24, pp. 1–14, Dec. 2021.
[23] M. Dryjanski and M. Szydelko, "A unified traffic steering framework for LTE radio access network coordination," *IEEE Commun. Mag.*, vol. 54, no. 7, pp. 84–92, Jul. 2016.
[24] M. A. Habib et al., "Traffic steering for 5G multi-RAT deployments using deep reinforcement learning," in *Proc. IEEE Consum. Commun. Netw. Conf. (CCNC)*, 2023, pp. 164–169.
[25] M. A. Habib et al., "Hierarchical reinforcement learning based traffic steering in multi-RAT 5G deployments," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2023, pp. 100–105.
[26] M. Jiang, "Device-controlled traffic steering in mobile networks," in *Proc. 9th Int. Conf. Next Gener. Mobile Appl., Services Techn.*, 2015, pp. 7–12.
[27] M. Zhang, X. Yang, T. Xu, and M.-T. Zhou, "Congestion-aware user-centric cooperative base station selection in ultra-dense networks," in *Proc. Int. Conf. Wireless Commun. Signal Process. (WCSP)*, Nanjing, China, 2017, pp. 1–6.
[28] K. Adachi, M. Li, P. H. Tan, Y. Zhou, and S. Sun, "Q-Learning based intelligent traffic steering in heterogeneous network," in *Proc. IEEE Veh. Technol. Conf. (VTC)*, 2016, pp. 1–5.
[29] A. Ben Mosbah, D. Griffith, and R. Rouil, "Enhanced transmission algorithm for dynamic device-to-device direct discovery," in *Proc. 15th IEEE Annu. Consum. Commun. Netw. Conf. (CCNC)*, 2018, pp. 1–8.
[30] Y. Türk, E. Zeydan, and C. A. Akbulut, "An experimental analysis of differentiated quality of service support for LTE users," in *Proc. 5th Int. Conf. Electr. Electron. Eng. (ICEEE)*, 2018, pp. 408–412.
[31] E. Li, L. Zeng, Z. Zhou, and X. Chen, "Edge AI: On-demand accelerating deep neural network inference via edge computing," *IEEE Trans. Wireless Commun.*, vol. 19, no. 1, pp. 447–457, Jan. 2020.
[32] D. Panno and S. Riolo, "A new joint scheduling scheme for GBR and non-GBR services in 5G RAN," in *Proc. IEEE 23rd Int. Workshop Comput.-Aided Model. Design Commun. Links Netw. (CAMAD)*, 2018, pp. 1–6.
[33] B. Liu, L. Wang, and M. Liu, "Lifelong federated reinforcement learning: A learning architecture for navigation in cloud robotic systems," *IEEE Robot. Autom. Lett.*, vol. 4, no. 4, pp. 4555–4562, Oct. 2019.
[34] V. Mnih et al., "Playing Atari with deep reinforcement learning," in *Proc. Workshop Deep Learn.*, 2013, pp. 1–9.
[35] X. Wang, R. Li, C. Wang, X. Li, T. Taleb, and V. C. M. Leung, "Attention-weighted federated deep reinforcement learning for device-to-device assisted heterogeneous collaborative edge caching," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 154–169, Jan. 2021.
[36] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.*, vol. 54, pp. 1273–1282, Feb. 2016.
[37] G. Long, M. Xie, T. Shen, T. Zhou, X. Wang, and J. Jiang, "Multi-center federated learning: Clients clustering for better personalization," 2020, *arXiv:2005.01026*.
[38] K. Akyol, "Comparing of deep neural networks and extreme learning machines based on growing and pruning approach," *Expert Syst. Appl.*, vol. 140, Feb. 2020, Art. no. 112875.
[39] H. G. Han, S. Zhang, and J. F. Qiao, "An adaptive growing and pruning algorithm for designing recurrent neural network," *Neurocomputing*, vol. 242, pp. 51–62, Jun. 2017.
[40] H. Hu, R. Peng, Y. W. Tai, and C. K. Tang, "Network trimming: A data-driven neuron pruning approach towards efficient deep architectures," 2016, *arXiv:1607.03250*.
[41] A. Lacava et al., "Programmable and customized intelligence for traffic steering in 5G networks using open RAN architectures," *IEEE Trans. Mobile Comput.*, early access, Apr. 13, 2023, doi: 10.1109/TMC.2023.3266642.



Han Zhang received the B.Eng. degree from Southeast University, China, in 2020. She is currently pursuing the Ph.D. degree with the University of Ottawa, Canada. She made a fast-track from the Master's program to the Ph.D. program with the University of Ottawa in 2021. She is devoted to applying deep reinforcement learning and federated learning techniques for 5G applications and security problems.



Hao Zhou (Member, IEEE) received the B.Eng. degree from the Huazhong University of Science and Technology, China, in 2016, the M.Eng. degree from Tianjin University, China, in 2019, and the Ph.D. degree from the University of Ottawa, Canada, in 2023. He is a future Postdoctoral Fellow with the Department of Computer Science, McGill University. His research interests include microgrid energy trading, 5G network slicing, mmWave, and reconfigurable intelligent surfaces. He is devoted to developing machine learning algorithms for smart grid and 5G/6G networks. He won the Best Paper Reward at the 2023 IEEE ICC Conference and the 2023 Outstanding Self-Financed Abroad Chinese Students Award.



Medhat Elsayed received the B.Sc. and M.Sc. degrees from Cairo University in 2009 and 2013, respectively, and the Ph.D. degree from the School of Electrical Engineering and Computer Science, University of Ottawa in 2021, where he also serves as an Adjunct Professor. He is a System Developer with Ericsson, Ottawa, Canada. His research interests include wireless networks (5G/6G), deep learning and reinforcement learning, and AI-enabled wireless networks.



Majid Bavand received the B.Sc. degree in electrical engineering from the Amirkabir University of Technology, Tehran, Iran, the M.Sc. degree in electrical engineering from Tarbiat Modares University, Tehran, Iran, and the Ph.D. degree in electrical and computer engineering from Queen's University, Kingston, Canada.

He is currently with Ericsson R&D, Ottawa, Canada. His interests include RAN architecture, cellular network evolution, and intelligent algorithm design for 5G NR with focus on beamforming, radio resource management, and energy efficiency.

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on October 05,2026 at 18:03:27 UTC from IEEE Xplore. Restrictions apply.



Raimundas Gaigalas received the Ph.D. degree in mathematical statistics from Uppsala University in 2004. Since 2005, he has been with Ericsson, working with the development of commercial features in various wireless RAN products with focus on concepts and simulators in Radio Resource Management Layer 2. AI/ML algorithms for RAN have been his main working area since 2017 and he is currently an AI/ML Developer with the Cloud RAN Unit, Ericsson.



Yigit Ozcan received the B.Sc. and M.Sc. degrees from the Electrical and Electronics Engineering Department, Middle East Technical University, Ankara, Turkey, in 2010 and 2013, respectively, and the Ph.D. degree from the Electrical and Computer Engineering Department, University of Waterloo, Waterloo, Canada, in 2019. He is currently working as a 5G System Developer with Ericsson. His research interests include resource allocation in wireless networks and networking protocols.



Melike Erol-Kantarci (Senior Member, IEEE) is a Chief Cloud RAN AI/ML Data Scientist with Ericsson and a Canada Research Chair of AI-enabled Next-Generation Wireless Networks and a Full Professor with the School of Electrical Engineering and Computer Science, University of Ottawa. She is the Founding Director of the Networked Systems and Communications Research (NETCORE) Laboratory. She has received numerous awards and recognitions. She is the Co-Editor of three books on *Smart Grids*, *Smart Cities* and *Intelligent Transportation*. She has over 200 peer-reviewed publications. She has delivered 70+ keynotes, plenary talks and tutorials around the globe. Her main research interests are AI-enabled wireless networks, 5G and 6G wireless communications, smart grid, and Internet of Things. She is on the editorial board of the IEEE TRANSACTIONS ON COMMUNICATIONS, IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING, and IEEE NETWORKING LETTERS. She has acted as the general chair and the technical program chair for many international conferences and workshops. She is an IEEE ComSoc Distinguished Lecturer and an ACM Senior Member.

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on October 05,2026 at 18:03:27 UTC from IEEE Xplore. Restrictions apply.